

附录 A 开发环境与方法

合适的开发环境和开发方法能大大提高编程的速度和正确性，但却常常被人忽视。本附录介绍命令行、脚本编程和编译器以及调试器的基本使用方法，希望能给读者带来帮助。

A.1 命令行

在图形用户界面（Graphical User Interface, GUI）日益发达的今天，命令行使用得越来越少。但笔者仍然认为命令行操作是每一位编程竞赛的选手必须掌握的技能。它不仅可以让你看起来很专业，而且确实能帮你很大的忙。

首先，进入命令行。在 Windows XP 中，可以选择“开始”菜单中的“运行”命令，在弹出的“运行”对话框中输入“cmd”，然后按 Enter 键，将出现类似下面的提示信息：

```
Microsoft Windows XP [版本 5.1.2600]  
(C) 版权所有 1985-2001 Microsoft Corp.
```

```
C:\Documents and Settings\Administrator>
```

其中，C:\Documents and Settings\Administrator 是当前路径，而后的“>”符号是命令提示符，紧跟其后的是闪烁的光标（cursor）。在文本界面中，所输入的任何信息都将出现在光标的所在位置。输入命令之后不要忘记按 Enter 键。

在 Linux 中，打开终端（terminal）即可进行命令行操作。Linux 终端并不一定会显示当前路径，可以用 pwd 命令将其显示。无论是 Windows 还是 Linux，都可以用上下箭头来翻阅并使用历史记录。Windows 和 Linux 下都可以用 Tab 键补全命令，但在细节上存在一些差异，读者可以自己实践或查阅相关资料。

A.1.1 文件系统

学习命令行的第一步是理解文件系统。相信读者对“文件”这一概念已经有所认识，但除此之外还需要清楚文件所在的位置。“位置”的表达方式有两种，一种是相对路径，另一种是绝对路径。

相对路径（relative path）是相对当前路径（current path）而言的，它在命令行中已有所体现。例如，在上面的例子中，当前路径是 C:\Documents and Settings\Administrator。在这种情况下，命令 type abc.txt 即为试图显示 C:\Documents and Settings\Administrator\abc.txt。

除了直接给出文件名外，还可以借助当前目录“.”和父目录“..”进行更为灵活的相对路径引用。例如，在上面的命令行提示符下输入 type..\..\Windows\123.txt，实际上是在试图

显示 `c:\Windows\123.txt`。

在命令行中可以用“`cd <目录名>`”的方式改变当前路径。例如，“`cd..`”会进入父目录，而“`cd aaa`”会进入当前目录的 `aaa` 子目录。

绝对路径和相对路径的区别是，前者给出了“起点”，其实际指向不随当前路径变化。在算法竞赛中，不要在提交的源代码中引用绝对路径，但在操作和调试程序的过程中可以随意使用绝对路径。另外，Linux 中的路径分隔符是正斜线“/”，而非反斜线“\”。

如果在程序中读写文件，则当前路径一般和该程序位于同一个目录，但也可以更改。如果在执行程序时出现“找不到文件”的错误，而文件确实存在，则极有可能是程序的“当前路径”与所想的相不一致。一个笨（但有效）的方法是用 `freopen("test.txt", "w", stdout)` 的方法创建文件 `test.txt`。找到了这个文件，就知道当前路径是什么了。如果要在 `freopen` 或者 `fopen` 中使用“`../../Windows/123.txt`”这样的相对路径，应注意反斜线字符在 C 语言的正确表示方法是“`\\`”。不过，即使在调试中也尽量不要使用路径名。如果在提交程序前忘记把路径名删除，将导致程序得 0 分。事实上，这样的例子并不少见。当然，如果只在条件编译中使用路径名，则是没有问题的。

最后一个小问题是：你不一定有存取文件的权限。如果出现类似于“`Permission Denied`”的错误信息，需确认当前用户是否拥有想访问的目录或者文件的访问/修改权。在现场比赛中，这可能是因为没使用比赛指定账户，而是改用 `guest` 登录了。

A.1.2 进程

简单地说，进程是一个程序正在执行时的实体。它消耗 CPU 资源且占用内存。进程一般都有名字，同时还有一个编号（称为 PID）。

在 Windows 和 Linux 中都能方便地列出进程。在 Windows 下可以使用 `Ctrl+Alt+Del` 组合键打开任务管理器，或者在命令行下用 `tasklist` 命令。在 Linux 下可以用 `top` 命令查看当前占用 CPU 资源最多的一些进程，而 `ps` 命令类似于 Windows 下的 `tasklist` 命令，它是使用列表的方式给出当前进程。在默认情况下，`ps` 命令并不会列出系统进程，用 `ps ax` 命令可以列出更多的进程。

强行终止进程有很多方法。在 Windows 下，可以用任务管理器直接终止，也可以在命令行下用 `taskkill /pid <PID>` 或 `taskkill /im <映像名>` 终止进程，可以通过执行 `taskkill /?` 查看更多选项。

在 Linux 下可以用 `kill` 命令终止命令，还可以用 `killall <进程名>` 命令把某个进程名对应的所有进程终止。一个典型情况是，如果 pascal 选手的 Lazarus IDE 不响应，就可以用 `killall lazarus` 把它们终止。

作为一个好习惯，当程序非正常终止，或者系统表现异常时，应检查进程。例如，若系统反应特别慢，可能是有一些看似运行结束，但其实残留在系统中继续占用系统资源的进程。

A.1.3 程序的执行

在命令行下执行一个程序比在 IDE 中执行要方便和灵活得多。基本的方法很简单：只

需直接输入程序名即可。

例如，在 Windows 下执行 `abc.exe`，可以进入它所在目录后直接输入 `abc` 并按 Enter 键。系统为什么能找到 `abc.exe` 呢，因为在 Windows 下，当前目录是最先搜寻可执行文件的位置，并且扩展名 `.exe` 在搜索之前会被自动添加。如果当前目录没有 `abc.exe`，是否会报错呢？不一定。运行 `path` 命令，会看到一连串目录。如果当前目录没有 `abc.exe`，系统会继续在这些目录中寻找，全部查找完毕仍没找到时才会报错。在搜索文件时并不会检查上述目录下的子目录。

Linux 有一些不同。首先，它的可执行文件名并不是以 `.exe` 为扩展名的，因此 `g++ abc.cpp -o abc` 编译出的文件是 `abc`，而非 `abc.exe`（当然，如果一定要将其取名为 `abc.exe`，也无不可）。另外，当前目录并不在搜索路径中，因此，即使 `abc` 已经在当前目录中，仍需要用 `./abc` 这样的方式告诉 Linux “可执行文件 `abc` 就在当前目录”。

A.1.4 重定向和管道

很多比赛要求选手直接读写标准输入输出（即用 `printf/scanf` 或 `cin/cout` 读写，且不用 `freopen`），难道在评分时裁判要将输入数据一一用键盘输入，等程序运行结束之后看着屏幕，逐个对照手中的标准答案吗？当然不是。可以使用重定向的技巧将输入文件塞到程序的标准输入中，然后再将程序输出保存在文件中。

在 Windows 下可以使用 `abc < abc.in > abc.out`。而在 Linux 下则可以使用 `./abc < abc.in > abc.out`。当然，如果可执行文件和输入输出文件不在同一个目录，则需要进行相应调整。但基本方法是不变的：在输入文件名前面加一个 `<` 符号，而在输出文件名前面加一个 `>` 符号。注意，此时的输出文件将被覆盖。如果希望只是把输出附加在文件末尾，则可用 `>>` 代替 `>`。此外，如果有大量的文本输出到标准错误输出，还可以用 `2>` 将它们重定向，但需注意，尽量不要在正式提交的程序中输出到标准错误输出，这样不仅可能会违反比赛规定，还可能会因为大量文本的输出而占用宝贵的 CPU 资源，甚至导致超时。

Windows 和 Linux 均提供“管道”机制，用于把不同的程序串起来。例如，如果有一个程序 `aplusb` 从标准输入读取两个整数 a 和 b ，计算并输出 $a+b$ ，还有一个程序 `sqr` 从标准输入读取一个整数 a ，计算并输出 a^2 ，则可以这样计算 $(10+20)^2$ ：`echo 10 20 | aplusb | sqr`。尽管也可以用重定向来完成这个任务，但用管道明显要简单得多。

另一个常见用法是分页显示一个文本文件的内容。在 Windows 下可以用 `type abc.txt | more`，在 Linux 下则是用 `cat abc.txt | more`。

A.1.5 常见命令

在 Linux 中，可以用 `time` 命令计时。例如，运行 `time ./abc` 会执行 `abc` 并输出运行时间。但 Windows 中并没有这样的命令，幸好在大多数情况下只是在对自己编写的程序计时，因此只需在程序的最后打印出 `clock() / (double)CLOCKS_PER_SEC` 即可（需要包含 `time.h`）。

附表 A-1 中给出了一些常见命令的 Linux 版本和 Windows 版本，供读者查阅。